

Application of Approximation Theory in Data Science

How classical mathematical concepts underpin modern machine learning?

Yahya Izadi

2025-12-30

1 Introduction

In the world of Data Science, we rarely have access to the true function generating the data ($f(x)$). What we have is a set of noisy observations, and our goal is to discover the hidden pattern in this data. **Approximation Theory**, as the beating heart of mathematical modeling, is the bridge between raw data and predictive models.

Approximation theory involves two general types of problems. One problem arises when a function is given explicitly, but we wish to find a “simpler” type of function, such as a polynomial, to approximate values of the given function. The other problem in approximation theory is concerned with fitting functions to given data and finding the “best” function in a certain class to represent the data.

This article examines three fundamental approximation approaches and their applications in machine learning:

1. **Discrete Least Squares:** The foundation of regression and Loss Functions.
2. **Continuous Least Squares:** The key to Feature Engineering and solving multicollinearity.
3. **Minimax Approximation:** The basis of algorithms like SVM and anomaly detection systems.

2 Discrete Least Squares: The “Hello World” of Machine Learning

While mathematicians view Least Squares as an approximation technique, in Data Science, it serves as the foundational architecture for **Supervised Learning**. It is the simplest form of “training” a model where we map inputs (Features) to outputs (Targets).

2.1 Mathematical Formulation: The Normal Equations

In a typical ML setting, we have a dataset with m training examples $\{(x_i, y_i)\}_{i=1}^m$. We want to learn a function $P(x)$ that predicts y . Let's define our hypothesis space as a polynomial of degree n :

$$P(x) = c_0 + c_1x + c_2x^2 + \dots + c_nx^n$$

In computer science, we don't solve this row by row. We vectorize the operation. We construct a **Design Matrix** (or Feature Matrix) A , where each row represents one data sample, and each column represents a feature:

$$A = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^n \\ 1 & x_2 & x_2^2 & \dots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & x_m^2 & \dots & x_m^n \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

Our goal is to solve $A\mathbf{c} \approx \mathbf{y}$. Since the system is overdetermined ($m \gg n$), no exact solution exists. We seek the vector $\mathbf{c}$ that minimizes the Sum of Squared Errors (SSE): $E = \|\mathbf{y} - A\mathbf{c}\|_2^2$.

The analytical solution that minimizes this error is given by the **Normal Equations**:

$$\mathbf{c} = (A^T A)^{-1} A^T \mathbf{y}$$

2.2 Why This Matters in Machine Learning?

For a Data Scientist, the equation above is not just algebra; it represents the core mechanics of learning:

1. **Feature Engineering (The Matrix A):** Notice that our raw input is just x . However, the matrix A contains columns like x^2 or x^3 . This is called **Basis Expansion**. We are projecting low-dimensional data into a higher-dimensional feature space to make it linearly separable.
2. **The Bias Term (c_0):** The first column of matrix A is entirely filled with **1s**. In Machine Learning, this corresponds to the **Bias** (or intercept). It allows the model to shift vertically. Without this column, our model would be forced to pass through the origin $(0, 0)$, which is a terrible constraint for most real-world data.
3. **Model Weights ($\mathbf{c}$):** The vector $\mathbf{c}$ represents the **Learned Parameters** (Weights).
 - If c_2 is large, it means the feature x^2 is very important for the prediction.
 - "Training" the model simply means calculating this vector.
4. **Overfitting vs. Underfitting:** The degree n (number of columns in A) is a **Hyperparameter**.
 - **Small n (e.g., $n = 1$):** The model is too simple (High Bias/Underfitting).
 - **Large n (e.g., $n = 15$):** The model memorizes noise (High Variance/Overfitting).

2.3 Practical Implementation: From Math to Code

Below is the implementation of these concepts. Note how we explicitly construct the feature matrix with the “Bias” column.

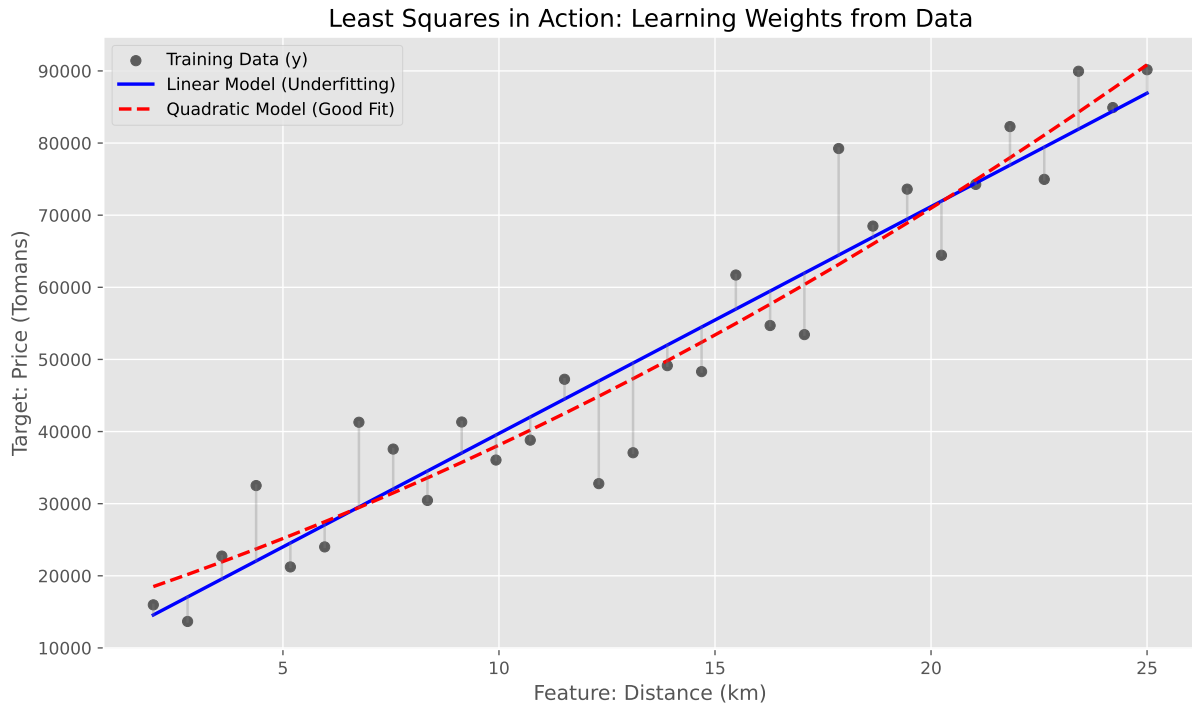


Figure 1: Linear and Quadratic Fit on Trip Price Data (Discrete Least Squares)

3 Continuous Least Squares and Feature Engineering

In the previous section, we treated data as discrete points. However, in advanced Machine Learning, we often view the underlying process as a continuous function. This shift leads us to one of the most sophisticated techniques in **Feature Engineering**.

When we perform Polynomial Regression, we are essentially creating new features $(x, x^2, x^3, \dots)$ from a single raw input x . This is a powerful technique to capture non-linearity, but it comes with a severe hidden cost in standard Machine Learning: **Multicollinearity**.

3.1 The Mathematical Foundation: Orthogonality

To understand the solution, we must look at the geometry of functions. In a vector space of functions, two functions $f(x)$ and $g(x)$ are considered “perpendicular” (or orthogonal) if their inner product is zero. Unlike vectors where we sum components, for continuous functions, we use an integral:

$$\langle f, g \rangle = \int_a^b f(x)g(x) dx = 0$$

The standard features we usually generate $(1, x, x^2, \dots)$ are **NOT** orthogonal. Mathematically, $\int_0^1 x \cdot x^2 dx \neq 0$. This means they “overlap” significantly.

To fix this, we change our basis to **Orthogonal Polynomials**, such as **Legendre Polynomials** ($L_n(x)$). These functions are constructed specifically to satisfy the orthogonality condition:

$$\int_{-1}^1 L_m(x)L_n(x) dx = 0 \quad \text{for } m \neq n$$

3.1.1 Visual Intuition: Why Must the Integral Be Zero?

The mathematical definition of orthogonality ($\int f(x)g(x)dx = 0$) often feels abstract. Why does a “zero integral” imply that two features are independent?

Think of the **Product** of two functions as a “Compatibility Check”:

- If both functions go up (positive) or both go down (negative), their product is **Positive** (Agreement).
- If one goes up and the other goes down, their product is **Negative** (Disagreement).

The **Integral** is simply the sum of all these agreements and disagreements across the entire range.

- **Standard Basis (e.g., x^2 vs x^3 on $[0, 1]$):** They are almost always in agreement (both positive). The integral is a large positive number, meaning they are highly correlated. The model sees them as “redundant”.
- **Orthogonal Basis (e.g., Legendre L_2 vs L_3):** They fluctuate. Sometimes they agree, sometimes they disagree. Crucially, the “positive agreements” perfectly cancel out the “negative disagreements”. The total sum (integral) is **Zero**.

This “Zero Balance” tells the Machine Learning model: *“These two features have absolutely no net relationship. You can trust the weight of one without worrying about the other.”*

Let’s visualize this “cancellation” effect:

3.2 Why This is Crucial for Machine Learning?

To understand the power of this method, let’s leave abstract math aside and solve a critical business problem.

The Scenario: Imagine you are a Data Scientist at a retail giant. Your task is to predict **Sales Revenue** based on the **Marketing Budget** (x).

We know this relationship is complex:

1. **Growth Phase:** Initially, spending more money leads to more sales (Linear effect).

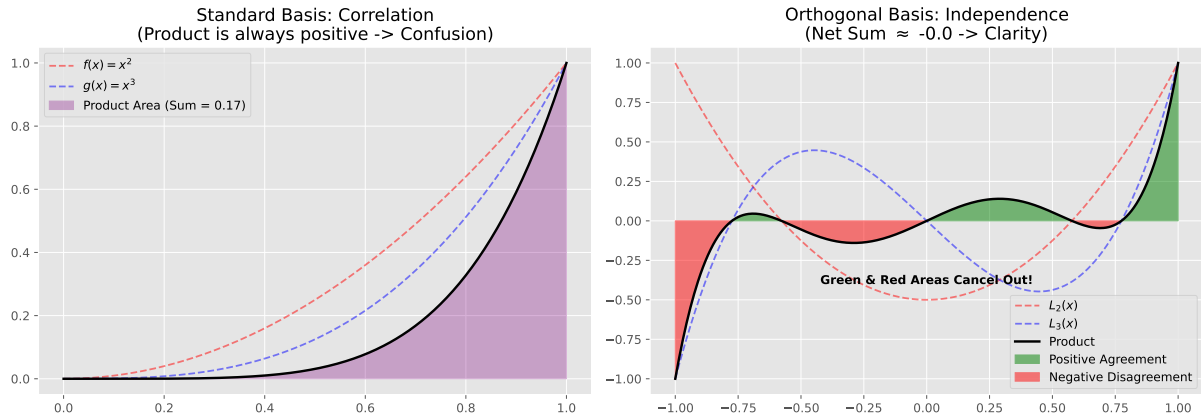


Figure 2: Visualizing Orthogonality: Why Areas Cancel Out

2. **Saturation Phase:** Eventually, the market gets saturated. Spending double the budget won't double the sales; the return diminishes (Curvature/Quadratic effect).

We need a polynomial model to capture this “Diminishing Returns”. Let’s compare the two approaches:

3.2.1 Solving the “Information Overlap” Problem

Suppose we use the standard approach and feed the model Budget (x) and Budget² (x^2).

- **The Problem:** In marketing data, when you increase the budget, both x and x^2 increase simultaneously. They are heavily correlated.
- **The ML Consequence (Confusion):** The model sees a change in sales, but it gets “confused”. Is the sales spike due to the raw spending power (x) or the market saturation curve (x^2)?
- **Result:** It splits the importance arbitrarily. You might get unstable coefficients that change wildly with just one new week of data.

3.2.2 Feature Independence and Stability

Now, imagine we use Orthogonal features (L_1, L_2).

- L_1 captures the “**Pure Growth Trend**”.
- L_2 captures the “**Pure Saturation Effect**”.
- **Scenario:** You decide to add a cubic term (L_3) to model a sudden viral trend.
- **Standard Failure:** In standard regression, adding x^3 would mess up your previous analysis of growth vs. saturation. All weights would shift.
- **Orthogonal Success:** Because L_3 is independent, adding it acts like a plug-and-play module. It adds new insight without destroying the stability of your previous findings.

3.2.3 Model Interpretability (The “White Box” Effect)

This is vital for explaining the model to the Marketing Director.

- **Standard Confusion:** “Mr. Director, the coefficient for Budget is 150, but for Budget-Squared is -0.05.” This is hard to translate into strategy.
- **Orthogonal Clarity:** The weights tell a clear story:
 - **High L_1 Weight:** “Our brand has strong fundamental growth.”
 - **High L_2 Weight:** “We are hitting the market saturation point very hard. We should stop increasing the budget.”
 - **Feature Selection:** If the L_2 weight is zero, you can confidently tell the team: “Don’t worry about saturation yet, keep spending!”

To visualize this power, let’s look at the simulation below. We decompose a noisy sales dataset into its constituent parts: the underlying growth trend and the market saturation effect. Notice how the orthogonal basis allows us to separate these effects cleanly.

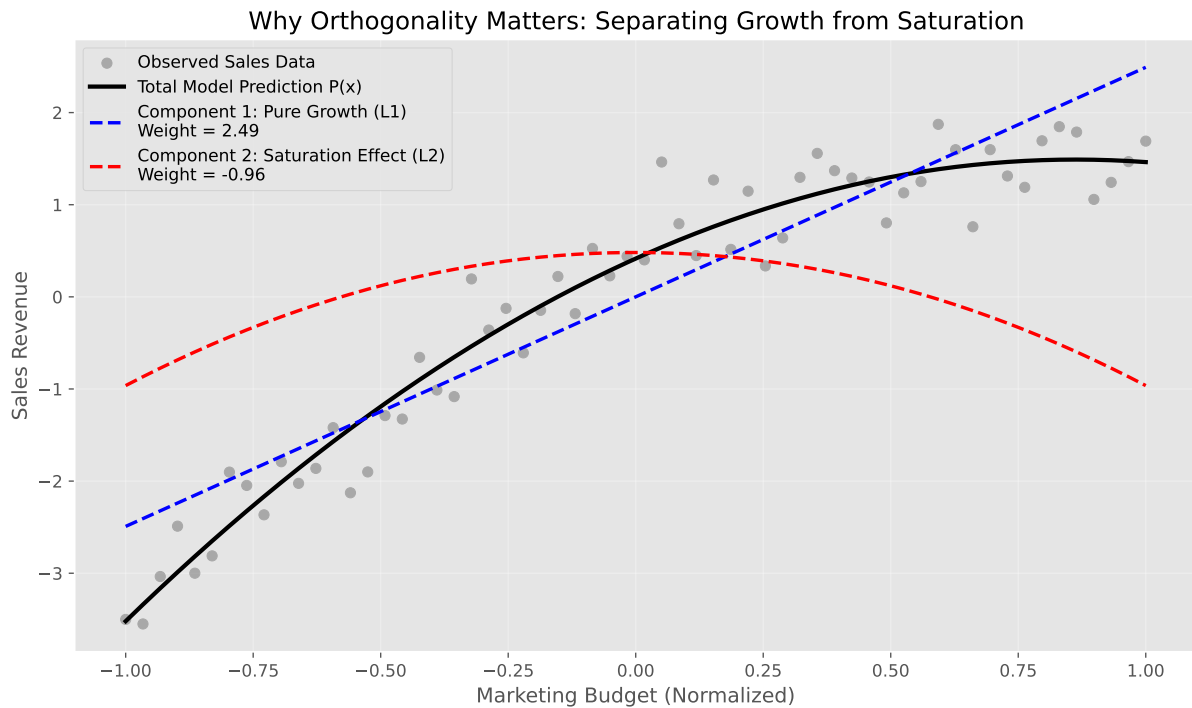


Figure 3: Decomposing Sales: Growth Trend vs. Market Saturation (Orthogonal Approach)

3.3 Practical Example: Visualizing Feature Correlation

Let’s verify this claim with data. We will compare the correlation matrix of the “Confused” standard model (Red) versus the “Clear” orthogonal model (Blue).

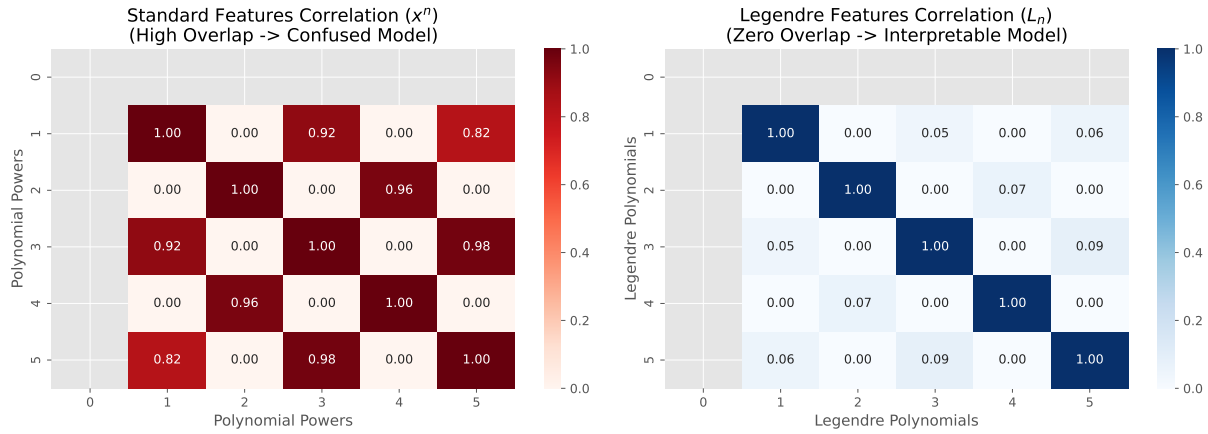


Figure 4: Feature Engineering: Standard Bases vs. Orthogonal Bases

3.3.1 Chart Analysis: Reading the Numbers

This heatmap is the “health report” of your feature set. The numbers range from **0 (No Correlation)** to **1 (Perfect Correlation)**.

- **Left Chart (Red - The Warning):**

- Look at the intersection of **Feature 2** (x^2) and **Feature 4** (x^4). The number is **0.96**.
- **What this means:** These two features are 96% identical! To the model, they look like twins. The model cannot mathematically distinguish which one is driving the prediction. This redundancy causes **Overfitting** and makes your model extremely sensitive to small noises in the data.

- **Right Chart (Blue - The Solution):**

- Look at the off-diagonal cells. The numbers are **0.00** or extremely close to it.
- **What this means:** Each feature is mathematically unique. Feature L_2 contains information that L_4 does not have, and vice versa. This guarantees that your model is **Stable**, **Efficient**, and the weights assigned to each feature are the true reflection of their importance.

4 Minimax Approximation: Managing Worst-Case Scenarios

While the “Least Squares” method (Section 1) seeks to satisfy the **majority** of data (democracy), the “Minimax” method focuses entirely on the **high-risk minority** or the hardest data.

In mathematics, this method is known as the **Infinity Norm** (L_∞ Norm). If we denote the error by e , the philosophical difference between these two methods is as follows:

- **Least Squares (L_2):** Tries to minimize $\sum e_i^2$. Meaning keeping the “average error of the whole system” low.
- **Minimax (L_∞):** Tries to minimize $\max |e_i|$. Meaning bringing the “worst single error” as low as possible.

This shift in perspective underpins three important categories of classical machine learning algorithms:

4.1 Support Vector Machines (SVM): A Minimax Masterpiece

The **SVM** algorithm is the most famous example of Minimax application in Classification.

Suppose you want to separate red and blue data with a line. There are infinite lines that do this. Which one is better?

- A line might pass through the middle of the data (Least Squares), but be very close to the reds.
- **Minimax Approach (SVM):** We don't care about data in the "heart" of the classes. We only care about data located on the **edges** (these are called support vectors).

The goal of SVM is: Find a line such that its distance to the nearest (most dangerous) data is maximized.

$$\text{Maximize} \left(\min_i \text{Distance}(x_i, \text{Line}) \right)$$

This is the concept of Margin. Maximizing the margin is equivalent to minimizing the risk of error in the worst case.

4.2 Location Problem: Public Welfare (K-Means) or Emergency Conditions (Minimax)?

One of the best ways to understand the difference between these two perspectives is the Clustering problem. Suppose we want to choose a few central points in a city. Two different strategies exist:

4.2.1 Strategy 1: Average Approach (K-Means Algorithm)

In this method, which is the most famous clustering algorithm, our goal is "majority comfort".

Example: Amazon wants to build 3 warehouses in Tehran.

Goal: Warehouses should be where the total distance traveled by couriers is minimized.

Result: Warehouses are concentrated in populated areas and the city center. If a customer lives on the outskirts of the city, the shipping cost for them is high, but since they are just "one person", they don't have much impact on the company's overall average. Here we have minimized the sum of squared errors (Variance).

4.2.2 Strategy 2: Minimax Approach (K-Centers Algorithm)

In this method, our goal is “covering the worst conditions”.

Example: The municipality wants to build 3 fire stations.

Goal: Here “average arrival time” is no longer important; rather, what matters is if the farthest house in the city catches fire, how long does it take for the fire truck to get there?

Result: Stations are distributed so that even the most remote points are within a standard radius. This might make the arrival time to the city center slightly longer, but it guarantees that no one waits more than 10 minutes.

Mathematics: Here we have minimized the distance of the farthest point to the center (Maximum Distance).

Key Note: K-Means does not care about outliers (it dissolves them in the average), but Minimax algorithms decide exactly based on those outliers and remote data.

4.3 Anomaly Detection

One of the vital applications of Minimax is in industry and cybersecurity. Suppose you have normal data (healthy bank transactions).

How do we draw a boundary around these to find fraudsters?

- We use a method called **Minimum Enclosing Ball**.
- Goal: Find the smallest circle that encloses **all** healthy data.
- The radius of this circle is determined by the **farthest** healthy data (Minimax).
- Any transaction that falls outside this “boundary of the worst healthy data” is flagged as an Anomaly.

4.4 Comparison Chart: When Minimax is a Lifesaver

For visual understanding, let’s see how badly the ordinary method performs and how excellent the Minimax method (Chebyshev nodes) is if the data has a specific pattern that fluctuates at the edges (Runge phenomenon).

4.4.1 Chart Analysis

In the chart above, red circles are drawn around points called **Support Vectors**. The entire mathematical model is built based only on these few points. Points deep inside the classes have no effect on the boundary line. This is the essence of the Minimax method: **Ignoring safe data and absolute focus on critical points**.

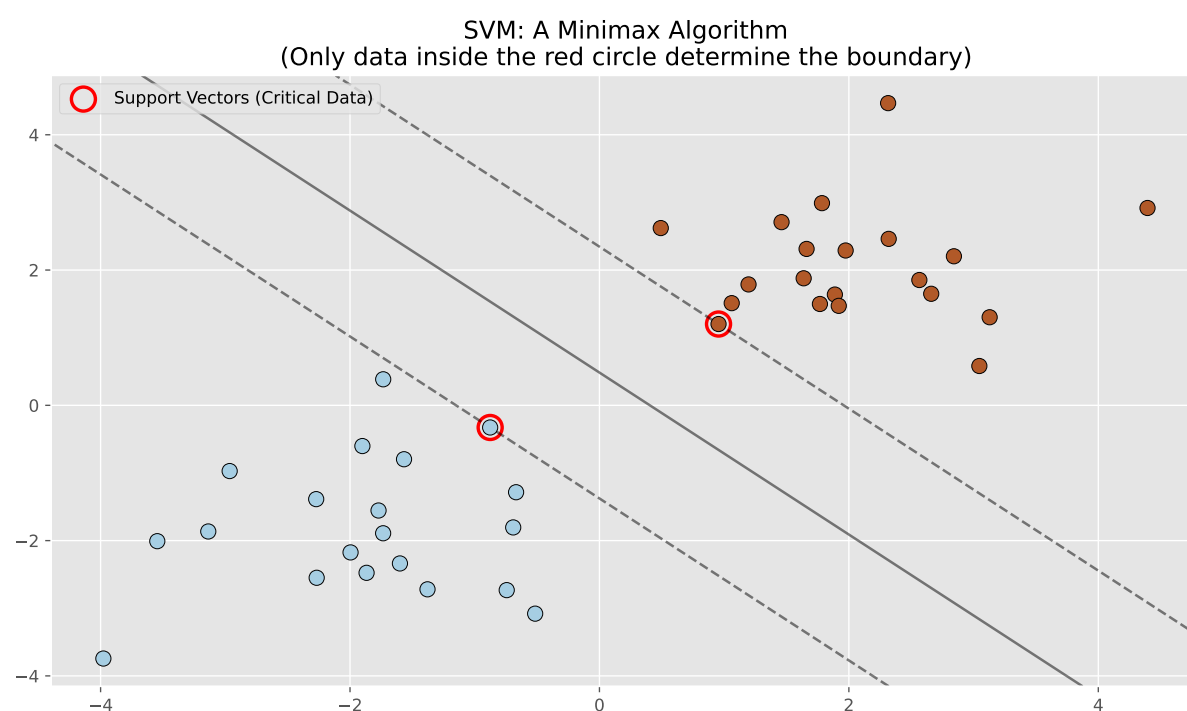


Figure 5: Philosophical Difference: Least Squares (Focus on Data Mass) vs. Minimax (Focus on Boundaries)